



ACRON Methodology Part-6

فاز 5: Cart Domain



در Part-4 ، فاز 4 تمام شد

حالا با قدرت وارد یکی از حساس‌ترین بخش‌های فروشگاه می‌شویم. سبد خرید در اپلیکیشن‌های مدرن (مثل دیجی‌کالا یا آمازون) تفاوت‌های زیادی با سیستم‌های قدیمی دارد.

تصمیمات معماری (چیستی و چرایی):

۱. آیا سبد خرید نیاز به لاگین دارد؟

در فروشگاه‌های مدرن، کاربر باید بتواند حتی بدون اینکه ثبت‌نام کرده باشد، محصولات را به سبد خرید اضافه کند. وقتی تصمیم به پرداخت گرفت، آن وقت لاگین می‌کند. پس ما نباید جدول **Cart** را به اجبار به **User** متصل کنیم.

۲. چرا از UUID به عنوان **Primary Key** (کلید اصلی) استفاده می‌کنیم؟

اگر آیدی سبد خرید عدد باشد (مثلاً `id=5`)، یک هکر به راحتی در Postman می‌نویسد `/api/carts/6/` و سبد خرید شخص دیگری را می‌بیند! وقتی کلید اصلی را UUID (یک رشته طولانی و تصادفی) قرار دهیم، حدس زدن سبد خرید دیگران عملاً غیرممکن می‌شود. فرانت‌اند این UUID را می‌گیرد و در **LocalStorage** مرورگر کاربر ذخیره می‌کند.

۳. جلوگیری از داده‌های تکراری (**Unique Together**):

اگر کاربر یک "لپ‌تاپ" در سبد خرید دارد و دوباره دکمه "افزودن به سبد" را می‌زند، دیتابیس

نباید دو ردیف جداگانه برای لپ‌تاپ بسازد! بلکه باید تعداد (Quantity) لپ‌تاپ را در همان ردیف قبلی از ۱ به ۲ تغییر دهد.

1- فعال‌سازی اپلیکیشن در `base.py`:

فایل `config/settings/base.py` را باز کنید و اپلیکیشن `carts` را از کامنت خارج کنید:

```
] = INSTALLED_APPS
... #
, 'apps.products'
, 'apps.carts' # این خط از کامنت خارج شود
[
```

2- طراحی ساختار دیتابیس در فایل `apps/carts/models.py`:

```
import uuid
from django.db import models
from apps.products.models import Product

class Cart(models.Model):
    # الف) جایگزینی ID عددی با UUID به عنوان
    # کلید اصلی امنیتی
    id = models.UUIDField(primary_key=True,
                          default=uuid.uuid4, editable=False)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return str(self.id)
```

```

class CartItem(models.Model)
# (ب) اتصال آیتم به سبد خرید
cart = models.ForeignKey(Cart, on_delete=
models.CASCADE, related_name='items')

# (ج) اتصال آیتم به محصول
product = models.ForeignKey(Product, on_
delete=models.CASCADE, related_name='cart_it
ems')

# (د) تعداد محصول در سبد
quantity = models.PositiveSmallIntegerFi
eld(default=1)

class Meta
# (هـ) جلوگیری از ساخت دو ردیف برای ی
ک محصول تکراری در یک سبد
unique_together = [['cart', 'produc
t']]

def __str__(self)
return f"{self.quantity} x {self.pro
duct.name}

```

کالبدشکافی عمیق کدها (چرایی):

- `id` جنگو به صورت پیش فرض یک فیلد عددی پنهان به نام `ID` در فیلد `primary_key=True` می سازد. با این دستور به دیتابیس می گوئیم: "ستون عددی پیش فرض را نسا، من دقیقاً همان کلید اصلی و ستون شماره یک جدول من باشد `UUID` می خواهم این
- اگر یک سبد خرید به هر دلیلی پاک شود، `cart` در رابطه * `on_delete=models.CASCADE` هم باید در دیتابیس دود شوند و از بین بروند. منطقی (`CartItem`) تمام آیتم های داخل آن نیست آیتمی در دیتابیس بماند که صاحب (سبد) ندارد.

- در رابطه `product`: اگر ادمین یک کالا را از سیستم حذف کند، آن کالا باید به صورت خودکار از سبد خرید تمام کاربران ناپدید شود.
- از این فیلد استفاده کردیم. چرا؟ (Quantity) برای تعداد کالا: `PositiveSmallIntegerField` چون تعداد کالا هرگز منفی نمی‌شود و از طرفی هیچکس ۳۲,۰۰۰ عدد لپ‌تاپ در یک سبد ساده، فضای کمتری در حافظه `Integer` به جای `SmallInteger` قرار نمی‌دهد! استفاده از RAM و هارد سرور اشغال می‌کند.
- MySQL این یک قفل ترکیبی در دیتابیس: `unique_together = [['cart', 'product']]` محصول `X` است. این کد به دیتابیس می‌گوید: "اگر کسی سعی کرد در سبد خرید شماره `Y` یاد Serializor بعداً در) کند، جلوی او را بگیر و ارور بده `INSERT` را برای بار دوم `Y` می‌گیریم که چطور این ارور را مدیریت کنیم و به جای ساختن ردیف جدید، عدد Quantity را `UPDATE` کنیم).

3- اعمال مدل‌ها در دیتابیس:

در ترمینال خود کدهای زیر را اجرا کنید تا جداول ساخته شوند:

```
python manage.py makemigrations carts
python manage.py migrate
```

خروجی:

```
$ python manage.py makemigrations carts
Migrations for 'carts':
  apps\carts\migrations\0001_initial.py
    + Create model Cart
    + Create model CartItem
```

```
$ python manage.py migrate
Operations to perform:
  Apply all migrations: accounts, admin, auth, carts, conte
nttypes, customers, products, sessions
Running migrations:
  Applying carts.0001_initial... OK
```




On-the-fly vs. Stored I

در این مرحله می‌خواهیم `CartSerializer` را بسازیم. اما قبل از اینکه دست به کیبورد ببریم، بیایید یک سناریوی مهم را در معماری نرم‌افزار بررسی کنیم: «محاسبه در لحظه» در برابر «ذخیره در دیتابیس» (On-the-fly vs. Stored). شاید این سوال پیش بیاید که چرا فیلدی به نام `total_price` را مستقیماً داخل مدل `CartItem` (در دیتابیس) ذخیره نکردیم؟ فرض کنید کاربر لپ‌تاپی به قیمت ۵۰ میلیون تومان را به سبد خرید اضافه می‌کند. اگر ما عدد ۵۰ میلیون را در رکورد سبد خرید ذخیره کنیم، چه می‌شود اگر فردا قیمت آن لپ‌تاپ در سایت تغییر کند و به ۵۵ میلیون برسد؟ سبد خرید کاربر همچنان ۵۰ میلیون را نشان می‌دهد که این یک باگ مالی خطرناک است! به همین دلیل، ما قیمت کل را در دیتابیس ذخیره نمی‌کنیم، بلکه در لحظه‌ای که کاربر سبد خرید را درخواست می‌کند، آن را به صورت زنده (Dynamic) محاسبه کرده و می‌فرستیم. این جادو در DRF با ابزاری به نام `SerializerMethodField` انجام می‌شود.



قدم اول: ساخت Serializer سبد خرید و آیتم‌ها

4- در مسیر `/apps/carts` یک فایل جدید به نام `serializers.py` بسازید و کدهای زیر را به دقت وارد کنید:

```
from rest_framework import serializers
from .models import Cart, CartItem
from apps.products.models import Product

# ۱. ساخت یک سریالایزر سبک و بهینه برای محصول
# داخل سبد خرید
class SimpleProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
```

```
# ما در سبد خرید به توضیحات طولانی یا  
برند نیازی نداریم، فقط اطلاعات حیاتی!  
fields = ['id', 'name', 'price', 'main_image']
```

```
# ۲. سریالایزر آیتم‌های داخل سبد (CartItem)  
class CartItemSerializer(serializers.ModelSerializer)
```

```
# نمایش اطلاعات محصول به صورت تو در تو، اما  
ما با سریالایزر سبد  
product = SimpleProductSerializer(read_only=True)
```

```
# تعریف یک فیلد محاسباتی که در دیتابیس وجود ندارد  
total_price = serializers.SerializerMethodField()
```

```
class Meta  
model = CartItem  
fields = ['id', 'product', 'quantity', 'total_price']
```

```
# متد متصل به فیلد محاسباتی بالا  
def get_total_price(self, cart_item: CartItem)  
# قیمت محصول × تعداد آن  
return cart_item.quantity * cart_item.product.price
```

```

۳. سریالایزر اصلی سبد خرید (Cart) #
class CartSerializer(serializers.ModelSerializer):
    id = serializers.UUIDField(read_only=True)

    # اتصال آیتم‌ها به سبد خرید (توجه: کلمه items همان related_name در مدل است)
    items = CartItemSerializer(many=True, read_only=True)

    # فیلد محاسباتی برای جمع کل مبلغ کل سبد خرید
    grand_total = serializers.SerializerMethodField()

    class Meta:
        model = Cart
        fields = ['id', 'items', 'grand_total']

    # متد متصل به فیلد محاسباتی مبلغ کل
    def get_grand_total(self, cart: Cart):
        # یک حلقه پایتونی برای جمع زدن قیمت کل تک تک آیتم‌های این سبد
        return sum([item.quantity * item.product.price for item in cart.items.all()])

```



کالبدشکافی عمیق و آموزش معماری این بخش

بیا این کدها را زیر ذره بین ببریم تا دقیقاً بفهمیم پشت صحنه چه اتفاقی می افتد:

۱. چرا `SimpleProductSerializer` را ساختیم؟ (مفهوم Over-fetching)
ما در فاز ۴ یک `ProductSerializer` بسیار قدرتمند ساختیم که دسته بندی، برند و گالری ۱۰ تایی تصاویر را لود می کرد. اگر همان را اینجا استفاده می کردیم، کاربر با باز کردن سبد خریدش حجم عظیمی از داده های غیرضروری را دانلود می کرد (به این مشکل در مهندسی نرم افزار Over-fetching می گویند که باعث کندی لود اپلیکیشن می شود).
با ساختن یک نسخه "ساده سازی شده" (فقط نام، عکس اصلی و قیمت)، سباز (Payload) پاسخ API را به شدت کاهش دادیم.

۲. مکانیزم `SerializerMethodField` چگونه کار می کند؟

وقتی شما فیلدی از این نوع تعریف می کنید (مثل `total_price`)، جنگو می فهمد که نباید به دنبال این ستون در دیتابیس بگردد. در عوض، به صورت خودکار به دنبال متدی می گردد که اسم آن با `get_` شروع شود و نام فیلد در ادامه آن بیاید (یعنی `get_total_price`).

• **پارامتر ورودی متد (`cart_item`):** DRF به صورت خودکار، همان رکورد دیتابیزی که در حال سریالایز شدن است را به این متد پاس می دهد.

- **Type Hinting (`cart_item: CartItem`):** این عبارت در پایتون اجباری نیست، اما یک (VS Code مثل) استاندارد کدنویسی بسیار تمیز است. با نوشتن آن، ویرایشگر کد شما است و وقتی نقطه (.) می گذارید، متدها `CartItem` می فهمد که این متغیر از جنس مدل `CartItem` است و فیلدهای آن را به شما پیشنهاد می دهد.

۳. جادوی محاسبه `grand_total` در پایتون

در متد `get_grand_total` ما از یک ویژگی قدرتمند پایتون به نام List Comprehension استفاده کردیم.

کد `[item.quantity * item.product.price for item in cart.items.all]` این کار را می کند:

- می رود تمام آیتم های این سبد را می آورد (`cart.items.all`).
- روی تک تک آن ها حلقه می زند.
- تعداد را ضربدر قیمت می کند.
- در نهایت یک لیست از اعداد (مبالغ) به ما می دهد (مثلاً `[120000, 50000]`).
- سپس تابع داخلی پایتون یعنی `sum()` تمام اعداد آن لیست را با هم جمع می کند و مبلغ نهایی فاکتور به دست می آید.

با این معماری، هر بار که کاربر API سبد خرید را صدا می زند، قیمت ها دقیقاً با آخرین قیمت های ثبت شده در دیتابیس محصولات محاسبه می شوند و هرگز مغایرتی پیش نخواهد

آمد.



حالا که این سریالایزر هوشمند را ساختیم، نوبت به ویوها (Views) می‌رسد. در View های سبد خرید، ما با یک چالش روبرو هستیم: "کاربر چطور یک آیتم را به سبد خرید اضافه کند در حالی که ما قانون `unique_together` (جلوگیری از کالای تکراری) را در دیتابیس گذاشته‌ایم؟"

همان‌طور که در دیتابیس تنظیم کردیم، یک سبد خرید نمی‌تواند دو ردیف مجزا برای یک محصول تکراری داشته باشد.

اگر کاربر یک «گوشی اپل» در سبد خود دارد و دوباره دکمه «افزودن به سبد» را می‌زند، ما نباید به دیتابیس دستور `INSERT` (ساخت ردیف جدید) بدهیم، زیرا دیتابیس خشمگین شده و خطای ۵۰۰ (Server Error) می‌دهد.

به جای آن، ما باید دستور `UPDATE` بدهیم و `quantity` (تعداد) را مثلاً از ۱ به ۲ تغییر دهیم. اما سریالایزرهای پیش‌فرض DRF این هوشمندی را ندارند. پس ما باید کنترل سریالایزر را به دست بگیریم.

این کار از طریق جداسازی `Serializer` های ورودی (Input) و خروجی (Output) و اورراید کردن متد `save` انجام می‌شود.



ساخت `Serializer` های ورودی (افزودن و ویرایش)

5- فایل `apps/carts/serializers.py` را باز کنید و کدهای زیر را به انتهای آن (زیر کدهای قبلی) اضافه کنید:

```
# این کلاس فقط برای زمانی است که کلاینت می‌خواهد محصولی را به سبد اضافه کند (POST)
class AddCartItemSerializer(serializers.ModelSerializer):
    # کلاینت فقط این دو شناسه را برای ما می‌فرستد
    product_id = serializers.IntegerField()
    cart_id = serializers.UUIDField()
```

```

class Meta
model = CartItem
fields = ['id', 'cart_id', 'product_
id', 'quantity']

# اعتبارسنجی: آیا این محصول اصلاً در فروشگاه
#   ماه وجود دارد؟
def validate_product_id(self, value)
if not Product.objects.filter(id=valu
:())ue).exists
raise serializers.ValidationError
("محصولی با این شناسه یافت نشد.")r
return value

# اعتبارسنجی: آیا این سبد خرید اصلاً وجود
#   دارد؟
def validate_cart_id(self, value)
if not Cart.objects.filter(id=valu
:())e).exists
raise serializers.ValidationError
("سبد خریدی با این شناسه یافت نشد.")r
return value

# قلب تپنده این بخش: تغییر رفتار ذخیره‌سا
#   زی برای دور زدن خطای دیتابیس
def save(self, **kwargs)
cart_id = self.validated_data['cart_
id']
product_id = self.validated_data['pr
oduct_id']

```

```

quantity = self.validated_data['quantity']

:try
    سناریو ۱: آیا این محصول از قبل
    در این سبد خرید وجود دارد؟
    cart_item = CartItem.objects.get
    (cart_id=cart_id, product_id=product_id)

    بله وجود دارد! پس فقط تعداد جد
    ید را با تعداد قبلی جمع کن
    cart_item.quantity += quantity
    ()cart_item.save

    self.instance = cart_item

:except CartItem.DoesNotExist
    سناریو ۲: محصول در سبد نیست. پ
    س یک ردیف جدید در دیتابیس بساز
    self.instance = CartItem.objects.create
    ,cart_id=cart_id
    ,product_id=product_id
    quantity=quantity
    (

    return self.instance

    # این کلاس فقط برای زمانی است که کلاینت میخواهد
    د تعداد را تغییر دهد (PATCH)

```



```
# مثلاً کاربر در سبد خرید دکمه + یا - را میزند
class UpdateCartItemSerializer(serializers.ModelSerializer):
    class Meta:
        model = CartItem
        fields = ['quantity']
```



کالبدشکافی کدهای Serializer (چیستی و چرایی)

- چرا کلاس‌های جدید ساختیم؟ در متدولوژی تمیز API، دیتایی که ما به کلاینت نشان می‌دهیم (خروجی) با دیتایی که از او می‌گیریم (ورودی) متفاوت است. `CartItemSerializer` (که در بخش قبل ساختیم) برای نمایش بود و شامل `product_name` و `total_price` بود. اما وقتی کلاینت می‌خواهد محصولی ثبت کند، فقط یک عدد `product_id` برای ما می‌فرستد. ما نمی‌توانیم از یک سریالایزر برای هر دو کار استفاده کنیم، بنابراین `AddCartItemSerializer` را فقط به عنوان "دروازه ورود داده" ساختیم.
- **متدهای** `<validate_<field_name>`: همانند فاز ۳، ما اینجا هم امنیت را بالا بردیم. اگر یک هکر آیدی محصولی را بفرستد که در دیتابیس نیست، سیستم کرش نمی‌کند، بلکه متد `validate_product_id` یک خطای تمیز 400 Bad Request برمی‌گرداند.
- **متد** `save` و **متغیر** `self.validated_data`: وقتی جنگو تمام ولیدیشن‌ها را با موفقیت پاس می‌کند، دیتاهای تایید شده را در دیکشنری `validated_data` قرار می‌دهد. ما در بلوک `try` تلاش می‌کنیم با دستور `get` آن کالا را در آن سبد پیدا کنیم. اگر پیدا شد، فقط فیلد `quantity` آپدیت می‌شود (دور زدن `unique_together`) و اگر پیدا نشد، خطای `DoesNotExist` رخ می‌دهد که ما آن را در بلوک `except` شکار می‌کنیم و با دستور `create` ردیف جدید را می‌سازیم.



قدم دوم: ساخت View‌های هوشمند سبد خرید

اکنون باید کنترلرهای API (ویوها) را بسازیم تا درخواست‌های کلاینت را دریافت کرده و به سریالایزرهای مناسب هدایت کنند.

6- در مسیر `/apps/carts` فایل `views.py` را ایجاد کرده و کدهای زیر را وارد کنید:

```

from rest_framework.viewsets import ModelViewSet, GenericViewSet
from rest_framework.mixins import CreateModelMixin, RetrieveModelMixin, DestroyModelMixin
from drf_spectacular.utils import extend_schema_view, extend_schema
from .models import Cart, CartItem
from .serializers import CartSerializer, CartItemSerializer, AddCartItemSerializer, UpdateCartItemSerializer

@extend_schema_view(
    create=extend_schema(
        summary="ساخت سبد خرید",
        tags=['Carts'],
    ),
    retrieve=extend_schema(
        summary="دریافت محتویات سبد خرید",
        tags=['Carts'],
    ),
    destroy=extend_schema(
        summary="حذف کامل سبد خرید",
        tags=['Carts'],
    ),
)
class CartViewSet(CreateModelMixin, RetrieveModelMixin, DestroyModelMixin, GenericViewSet):
    """
    ویو برای مدیریت خود سبد خرید (بدون آیتمها).
    توجه: متد List حذف شده است زیرا هیچ کاربری نباید لیست سبد خرید دیگران را ببیند.
    """
    # بهینه‌سازی کوئری دیتابیس برای جلوگیری از

```

ز مشکل N+1 در دریافت آیتم‌های سبد

```

queryset = Cart.objects.prefetch_related
('items__product').all
serializer_class = CartSerializer

)extend_schema_view@
create=extend_schema
(tags=['Cart Items'], "ول به سبد خرید",
partial_update=extend_schema
(tags=['Cart Item', "ییر تعداد یک محصول در سبد",
(s']
destroy=extend_schema
(tags=['Cart Items'], "صول از سبد خرید",
(
: class CartItemViewSet(ModelViewSet)
"""
ویو برای مدیریت آیتم‌های داخل سبد خرید.
"""
# جلوگیری از استفاده از متد PUT (ما فقط
به PATCH برای تغییر تعداد نیاز داریم)
http_method_names = ['post', 'patch', 'd
elete']

queryset = CartItem.objects.select_relat
('product').all

# جادوی DRF: انتخاب سریالایزر به صورت دین
امیک بر اساس نوع درخواست (Method)
: def get_serializer_class(self)
: if self.request.method == 'POST

```

```

return AddCartItemSerializer
: 'elif self.request.method == 'PATCH
return UpdateCartItemSerializer

return CartItemSerializer

```



کالبدشکافی کدهای View

- چرا `CartViewSet` از `ModelViewSet` ارث‌بری نکرد؟
یک `ModelViewSet` کامل، ۵ متد دارد: ساختن، خواندن تکی، آپدیت کردن، پاک کردن و لیست کردن همه (`List`). اگر ما از `ModelViewSet` استفاده می‌کردیم، مسیر `GET` `//api/carts` باز می‌شد و هرکسی می‌توانست تمام سبدهای خرید فروشگاه را ببیند (یک فاجعه امنیتی!). با ارث‌بری از `GenericViewSet` و تزریق فقط سه میکسین (`Mixin`) ضروری یعنی `Create`, `Retrieve`, `Destroy`، ما به صورت جراحی‌گونه متدهای خطرناک را مسدود کردیم.
- علامت `__` (دو آندرلین) در جنگو به معنای نفوذ به : `prefetch_related('items__product')` عمق جداول است. این کد می‌گوید: "وقتی سبد خرید را از دیتابیس آوردی، تمام (Cache) را هم از قبل کش (`product`) و تمام محصولات آن آیتم‌ها (`items`) آیتم‌هایش". برنخوریم N+1 کن تا به مشکل
- متد `get_serializer_class`: این یکی از پیشرفته‌ترین تکنیک‌های DRF است (Dynamic Serializers). کلاینت به یک URL واحد متصل می‌شود، اما بک‌اند ما می‌فهمد که اگر درخواست `POST` بود، باید از سریالایزر `AddCartItem` (که فقط آی‌دی می‌گیرد) استفاده کند و اگر `PATCH` بود از سریالایزر آپدیت استفاده کند.



قدم سوم: تنظیم URL‌های روتر

7- در مسیر `/apps/carts` فایل `urls.py` را ایجاد کنید:

```

from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import CartViewSet, CartItemView

```

Set

```
()router = DefaultRouter
# ثبت ویوست سبد خرید (آیدی این مسیر از نوع U
# (UID خواهد بود)
router.register('carts', CartViewSet, basena
me='carts')

# ثبت ویوست آیتم‌های سبد خرید
router.register('cart-items', CartItemViewSe
t, basename='cart-items')

] = urlpatterns
,path('', include(router.urls))
[
```

8- مسیر نهایی را در `apps/api/urls.py` اضافه کنید:

```
# بخشی از فایل apps/api/urls.py
] = urlpatterns
... #
path('customers/', include('apps.custome
,rs.urls'))
path('products/', include('apps.product
,s.urls'))
# اضافه کردن مسیر سبد خرید به قلب API
,path('', include('apps.carts.urls'))
[
```

(نکته: چون در روتر خودش کلمات `cart-items` و `cards` را تعریف کردیم، اینجا رشته را خالی گذاشتیم `''` تا URL ها دابل نشوند).

تست عملیات (مهم ترین بخش)

حالا سرور را روشن کنید و به آدرس Swagger

(`http://127.0.0.1:8000/api/docs/`) بروید.

۱. یک درخواست `POST` به `/api/cards/` بزنید تا یک سبد خرید خالی ساخته شود. UUID تولید شده را کپی کنید.

۲. حالا یک درخواست `POST` به `/api/cart-items/` بزنید. در Body درخواست، UUID سبد خرید، آی دی یک محصول از فروشگاه، و تعداد (`quantity: 1`) را بفرستید.

۳. تست جادوی ما: دوباره همان درخواست را عیناً تکرار کنید. خواهید دید که به جای ارور دیتابیس، تعداد محصول شما ۲ می شود!

در اینجا راهنمای قدم به قدم برای ویرایش و ارسال Body در Swagger آمده است:

راهنمای تصویری و قدم به قدم ارسال Body در Swagger

1- پیدا کردن Endpoint مورد نظر

- در صفحه Swagger (آدرس `http://127.0.0.1:8000/api/docs`)، به پایین اسکرول کنید تا به تگ **Cart Items** (که در `extend_schema@` تعریف کردیم) برسید.
- روی نوار سبز رنگ که نوشته شده `POST /api/cart-items/` (افزودن محصول به سبد خرید) کلیک کنید تا این بخش باز و منبسط شود.

2- فعال سازی حالت تست

- در گوشه سمت راست پنلی که باز شد، یک دکمه خاکستری یا سفید رنگ به نام **Try it** وجود دارد. روی آن کلیک کنید.
- چرایی: با زدن این دکمه، Swagger فرم ها و باکس های متنی را از حالت "فقط خواندنی" (Read-only) خارج کرده و به شما اجازه ویرایش و ارسال اطلاعات به سرور را می دهد.

3- پیدا کردن و ویرایش باکس Body

- بعد از زدن دکمه **Try it out**، کمی پایین‌تر بخشی به نام **Request body** ظاهر می‌شود.
- داخل این بخش، یک باکس متنی مشکی یا خاکستری رنگ (شبه به محیط کدنویسی) می‌بینید که حاوی یک ساختار JSON اولیه است.
- شما باید اطلاعات داخل این باکس را پاک کنید و اطلاعات واقعی خودتان را به شکل زیر جایگزین کنید:

```
{
  "cart_id": "UUID-خریدی-که-در-مرحله-۱-کپی-کردید-UUID",
  "product_id": 1,
  "quantity": 1
}
```

(نکته: به جای 1 در **product_id**، آیدی یکی از محصولات که قبلاً در پنل ادمین ساخته‌اید را قرار دهید. دقت کنید که ساختار JSON با دقت رعایت شود، مثلاً کلمات داخل کوتیشن "" باشند).

4- ارسال درخواست به سمت بک‌اند (Execute)

- دقیقاً در زیر همان باکس Request body، یک دکمه بزرگ آبی رنگ به نام **Execute** وجود دارد. روی آن کلیک کنید.
- چرایی: این دکمه دقیقاً معادل دکمه Send در Postman است. با زدن آن، مرورگر شما درخواست POST را به همراه بدنه JSON که نوشتید، به وبوی **CartItemViewSet** در بک‌اند جنگو می‌فرستد.

5- مشاهده نتیجه (Responses)

- کمی پایین‌تر از دکمه Execute، بخشی به نام **Responses** و قسمت **Server response** وجود دارد.
- در آنجا می‌توانید **Code** (که باید 201 یا 200 باشد) و **Details** (که همان خروجی JSON از سمت سرور است) را مشاهده کنید.

6- حالا دقیقاً همین عملیات را انجام دهید، یک بار محصول را ثبت کنید و سپس بدون اینکه چیزی را تغییر دهید، دوباره دکمه **Execute** را بزنید. در پاسخ دوم سرور، باید ببینید که اروری دریافت نمی‌کنید و عدد **quantity** برای آن محصول از ۱ به ۲ تغییر کرده است. این همان جادویی است که با اورراید کردن متد **save** در **Serializer** پیاده‌سازی کردیم!

تصمیم معماری و راه حل

همان طور که قبلاً بحث کردیم، سبب خرید باید برای کاربران مهمان (بدون لاگین) هم باز باشد. شاید بپرسید آیا این کار از نظر امنیتی خطرناک نیست؟

پاسخ: خیر. چون کلید اصلی سبب خرید ما یک **UUID** است (مثلاً `3fa85f64-5717-4562-b3fc-2c963f66afa6`). حدس زدن این رشته تصادفی برای هکرها عملاً غیرممکن است (احتمال ۱ در 2^{122}). بنابراین فقط کسی که این **UUID** را دارد (که در مرورگر خود کاربر ذخیره می‌شود) می‌تواند سبب خرید را ببیند یا ویرایش کند.



قفل‌گذاری از API های سبب خرید:

9- فایل `apps/carts/views.py` را باز کنید. ماژول `AllowAny` را ایمپورت کرده و آن را به هر دو کلاس اضافه کنید: لطفاً از کپی و پیست استفاده نکنید!!! تمام کد را خوانده و قسمت های لازم را انتقال دهید:

```
# این ایمپورت را به بالای فایل اضافه کنید
from rest_framework.permissions import AllowAny

... کدهای قبلی ...

class CartViewSet(CreateModelMixin, RetrieveModelMixin, DestroyModelMixin, GenericViewSet):

    # این خط را اضافه کنید تا قفل شکسته شود
    permission_classes = [AllowAny]

    queryset = Cart.objects.prefetch_related(
        ('items__product')).all()
    serializer_class = CartSerializer
```



```
:class CartItemViewSet(ModelViewSet)
# این خط را اضافه کنید تا قفل شکسته شود
permission_classes = [AllowAny]

http_method_names = ['post', 'patch', 'delete']
queryset = CartItem.objects.select_related('product').all

... # بقیه کدها بدون تغییر ...
```



یک نکته آموزشی درباره Swagger (قفل بالای صفحه)

- اگر در آینده API خاصی داشتید که واقعاً نیاز به لاگین داشت (مثل پروفایل مشتری) و خواستید آن را در Swagger تست کنید، باید چه کار کنید؟
 1. ابتدا از طریق مسیر `/POST /api/token` یوزرنیم و پسورد خود را می‌فرستید.
 2. سرور به شما یک `access_token` می‌دهد. آن را کپی می‌کنید.
 3. در بالاترین قسمت صفحه Swagger، یک دکمه سبز رنگ با آیکون قفل به نام **Authorize** وجود دارد. روی آن کلیک می‌کنید.
 4. توکن خود را در باکس مربوطه Paste می‌کنید و دکمه **Authorize** را می‌زنید.
- از این به بعد، Swagger به صورت خودکار توکن شما را در تمام درخواست‌هایی که با دکمه **Execute** می‌زنید، تزریق می‌کند و دیگر خطای ۴۰۱ نمی‌گیرید.



پایان Part-6